

NCTU-EE ICLAB – Fall 2025

Lab05 Exercise

Design: H.264 Lite Prediction and Transform Engine (HLPTE)

Data Preparation

- Extract the test data from TA's directory:
% tar xvf ~iclabTA01/Lab05_2025F.tar
- The extracted LAB directory contains:
 - Practice/
 - Exercise/

Design Description

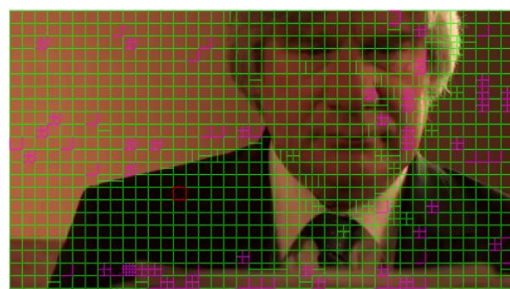
H.264, also known as AVC (Advanced Video Coding), is a widely used video compression standard developed to efficiently represent video data with reduced storage and transmission requirements. The main goal of H.264 is to eliminate redundancy in video sequences while preserving high visual quality.

To achieve this, H.264 uses spatial prediction (intra prediction) to exploit correlations within a single frame, and temporal prediction (inter prediction) to exploit similarities across successive frames.

- Intra prediction reduces spatial redundancy by predicting pixel values based on neighboring pixels within the same frame.
- Inter prediction reduces temporal redundancy by predicting the current frame's content from one or more reference frames, using motion estimation and motion compensation techniques. This involves finding matching blocks between frames and representing them with motion vectors and residual differences.



Intra Prediction



Inter Prediction

Figure 1. Intra prediction and inter prediction of H.264

The differences, or residuals, between the original data and the predictions are then compressed further using integer transforms, quantization, and entropy coding.

Through this combination of prediction and efficient coding, H.264 achieves significantly higher compression efficiency compared to earlier standards such as MPEG-2, allowing high-quality video delivery at much lower bitrates. This makes H.264 suitable for a wide range of applications, from video conferencing and streaming to Blu-ray discs and broadcast television.

In this exercise, you need to build a design to perform H.264 Lite Prediction and Transform Engine (HLPTE) with several parameters. For the input frames require large space to store, you are asked to use memory for finishing this lab.

- Algorithm Overview of H.264 Prediction and Transform Engine

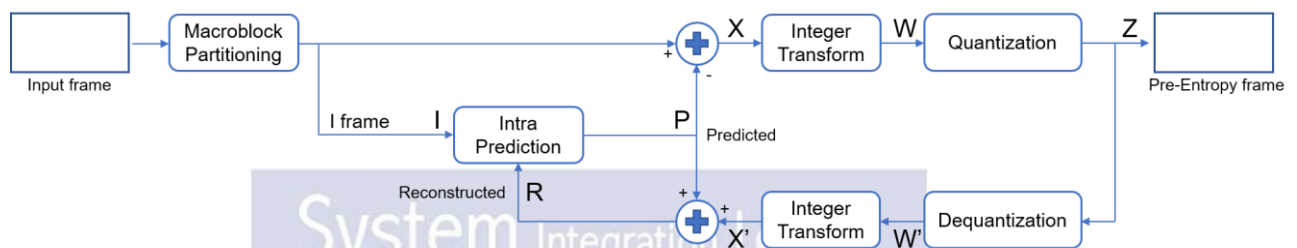


Figure 2. Block diagram of H.264 Pre-Entropy Encoder

- Macroblock Partitioning

The input frame is partitioned into **16×16 macroblocks** (MBs) based on its height and width, which serve as the basic processing units in H.264 encoding. For example, if the input frame has a resolution of 144×256 pixels, it will be divided into 144 macroblocks.

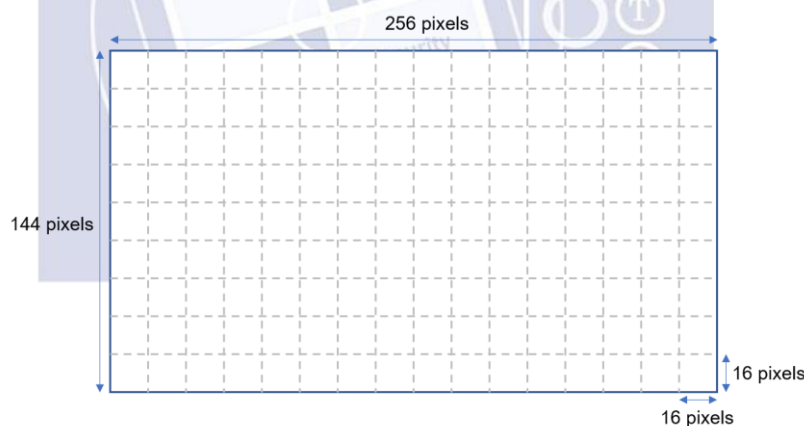


Figure 3. Example for Macroblock Partitioning

- Intra Prediction

Intra prediction exploits spatial correlation within a single frame by using **already reconstructed neighboring pixels** to generate a prediction for the current block. Two types of intra prediction modes are considered:

- Intra 4x4 Prediction

In the Intra 4×4 mode, the 16×16 macroblock is first divided into 16 4×4 blocks. Each block is predicted from the **already reconstructed neighboring pixels** (top and/or left). For simplicity, only three prediction modes are considered:

■ DC mode

The 4x4 predicted block is generated by taking the average of the **available reconstructed neighboring pixels** (top and left).

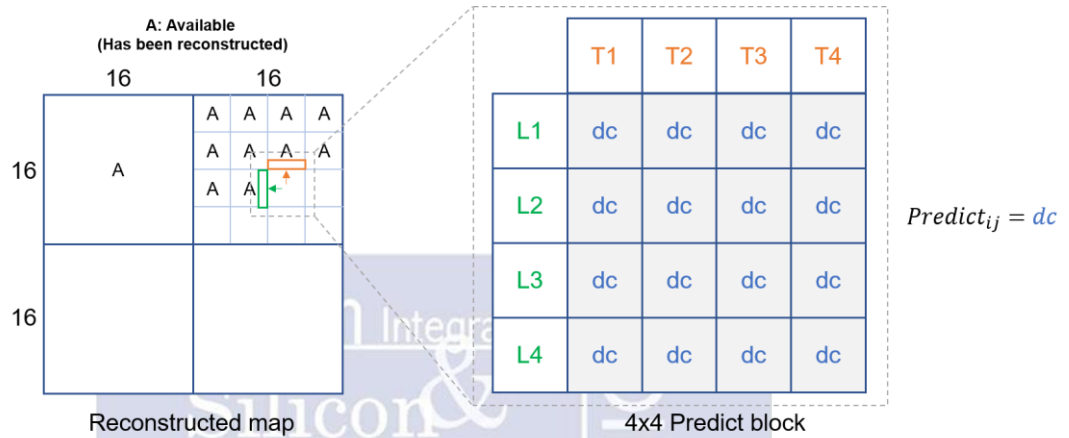


Figure 4. DC mode for Intra 4x4 prediction

The calculation of dc value is listed as following formula:

$$dc = \begin{cases} \text{sum}(T + L) \gg 3 & \text{if both } T \text{ and } L \text{ are available} \\ \text{sum}(T) \gg 2 & \text{if only } T \text{ are available} \\ \text{sum}(L) \gg 2 & \text{if only } L \text{ are available} \\ 128 & \text{if neither } T \text{ nor } L \text{ are available} \end{cases}$$

■ Horizontal mode

The 4x4 predicted block is generated by replicating the **available reconstructed neighboring pixels** from the block on the left.

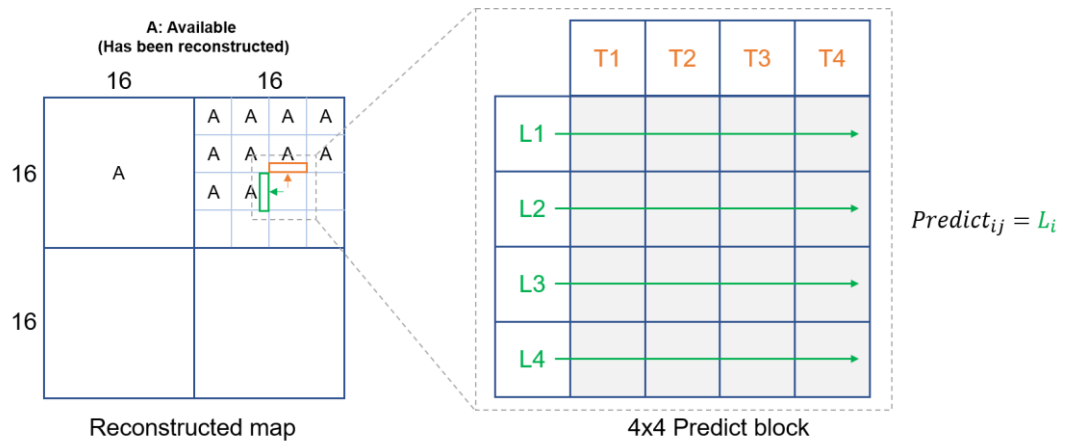


Figure 5. Horizontal mode for Intra 4x4 prediction

■ Vertical mode

The 4x4 predicted block is generated by replicating the **available reconstructed neighboring pixels** from the block above.

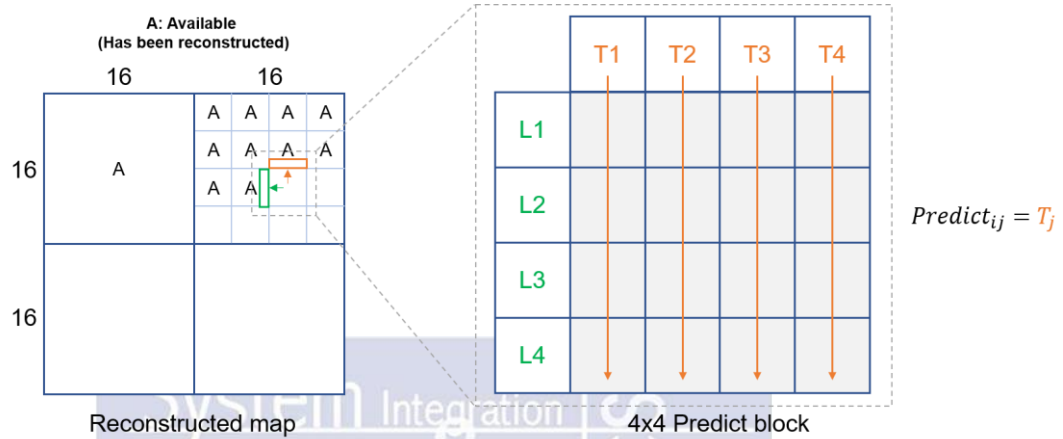


Figure 6. Vertical mode for Intra 4x4 prediction

■ Sum of absolute Difference (SAD)

SAD is a distortion metric used to evaluate the accuracy of intra prediction. It is computed as the sum of absolute differences between the input block and the predict block:

$$SAD = \sum_{i=0}^3 \sum_{j=0}^3 |input(i, j) - predict(i, j)|$$

■ How to determine predicted block

For each 4×4 block, follow these rules when selecting prediction modes:

1. If **both left and top pixels are available**, evaluate **DC**, **Horizontal**, and **Vertical** modes.
2. If **only left pixels are available**, evaluate **DC** and **Horizontal** modes.
3. If **only top pixels are available**, evaluate **DC** and **Vertical** modes.
4. If **neither left nor top pixels are available**, evaluate **only DC** mode.

The prediction mode with the **smallest SAD (Sum of Absolute Differences)** is considered the best. If two or more modes yield the same SAD value, select the mode according to the following priority order: **DC > Horizontal > Vertical**.

➤ Intra 16x16 prediction

Each 16x16 macroblock block is predicted from the **already reconstructed neighboring pixels** (top and/or left). For simplicity, only three prediction modes are considered:

■ DC mode

The 16x16 predicted block is generated by taking the average of the **available reconstructed neighboring pixels** (top and/or left).

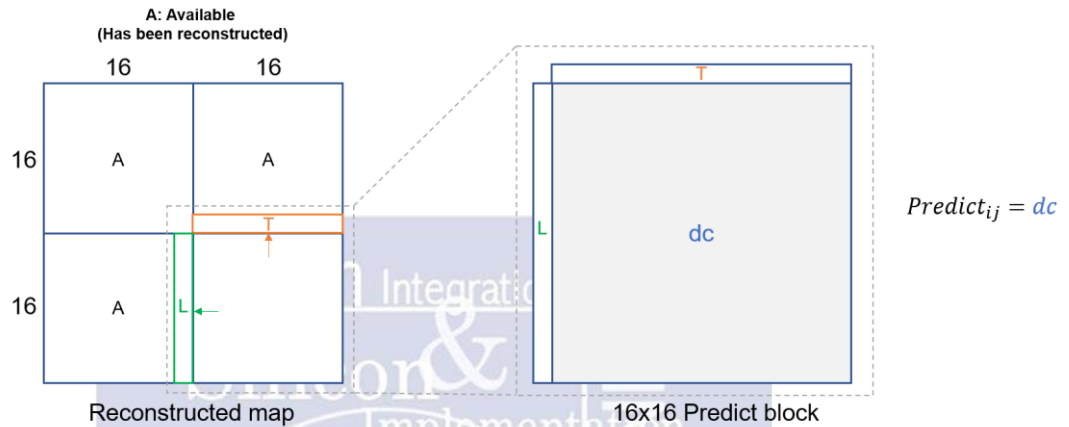


Figure 7. DC mode for Intra 16x16 prediction

The calculation of dc value is listed as following formula:

$$dc = \begin{cases} \text{sum}(T + L) \gg 5 & \text{if both } T \text{ and } L \text{ are available} \\ \text{sum}(T) \gg 4 & \text{if only } T \text{ are available} \\ \text{sum}(L) \gg 4 & \text{if only } L \text{ are available} \\ 128 & \text{if neither } T \text{ nor } L \text{ are available} \end{cases}$$

■ Horizontal mode

The 16x16 predicted block is generated by replicating the **available reconstructed neighboring pixels** from the block on the left.

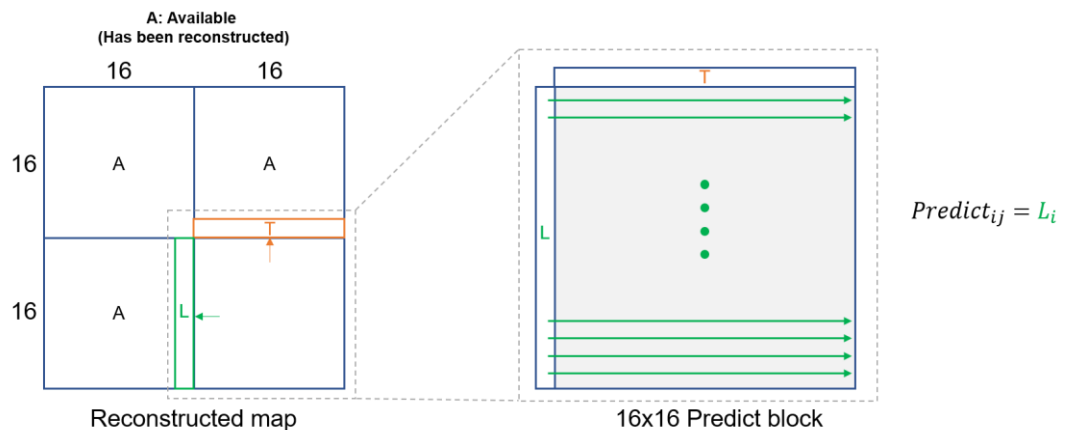


Figure 8. Horizontal mode for Intra 16x16 prediction

■ Vertical mode

The 16x16 predicted block is generated by replicating the **available reconstructed neighboring pixels** from the block above.

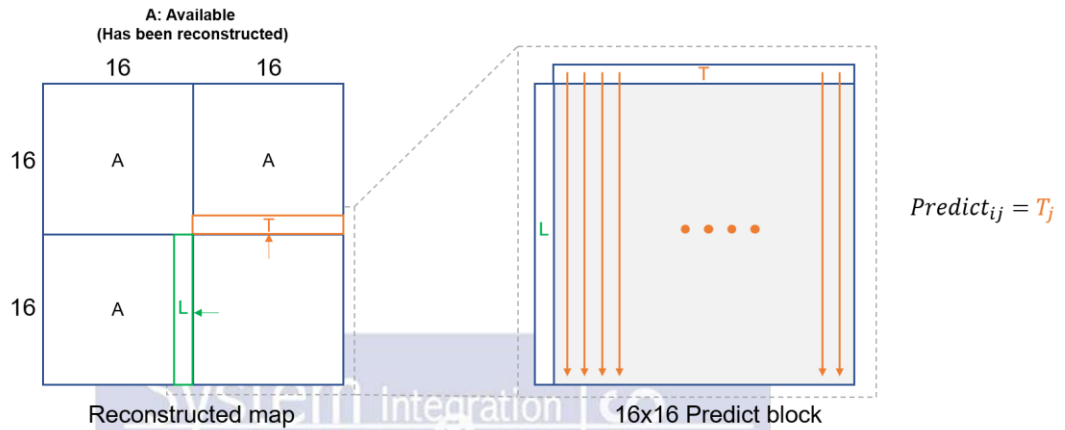


Figure 9. Vertical mode for Intra 16x16 prediction

■ Sum of absolute Difference (SAD)

SAD is a distortion metric used to evaluate the accuracy of intra prediction. It is computed as the **sum of absolute differences between the input block and the predict block**:

$$SAD = \sum_{i=0}^{15} \sum_{j=0}^{15} |input(i, j) - predict(i, j)|$$

■ How to determine predicted block

For each 16×16 macroblock, follow these rules when selecting prediction modes:

1. If **both left and top pixels are available**, evaluate **DC**, **Horizontal**, and **Vertical** modes.
2. If **only left pixels are available**, evaluate **DC** and **Horizontal** modes.
3. If **only top pixels are available**, evaluate **DC** and **Vertical** modes.
4. If **neither left nor top pixels are available**, evaluate **only DC** mode.

The prediction mode with the **smallest SAD (Sum of Absolute Differences)** is considered the best. If two or more modes yield the same SAD value, select the mode according to the following priority order: **DC > Horizontal > Vertical**.

- Residual Computation

After intra prediction, the predicted block is subtracted from the original block to obtain the **residual block**. The residual represents the difference between the original pixels and the predicted pixels, and it contains the information that cannot be captured by prediction alone.

$$Residual(i, j) = Input(i, j) - Predicted(i, j) \text{ for each pixel}$$

- Integer Transform

After the prediction process, the **residual block (X)** is transformed using a **4×4 integer transform**. This step converts the spatial domain residuals into frequency domain coefficients, allowing energy to be compacted mainly into the low-frequency components. **If Intra 16×16 prediction is performed, the 16×16 residual block must first be divided into sixteen 4×4 blocks, execute this operation for each 4x4 block and then combine them into 16x16 transformed block (W).** The formula is shown as below:

$$W = C_f X C_f^T, \text{ where } C_f = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \end{bmatrix}$$

- Quantization

After the integer transform, the **transformed block (W)** is quantized to reduce their precision and achieve compression. This is done by dividing each coefficient by a quantization step size determined by the **quantization parameter (QP)** and rounding the result. Larger QP values lead to coarser quantization (higher compression but lower quality), while smaller QP values preserve more details (lower compression but higher quality). **If Intra 16×16 prediction is performed, the 16×16 transformed block must first be divided into sixteen 4×4 blocks, execute this operation for each 4x4 block and then combine them into 16x16 pre-entropy block (Z).** The formula is shown as below:

The value and sign of Z_{ij} are calculated independently

$$|Z_{ij}| = (|W_{ij}| \cdot MF_{ij} + f) \gg qbits, \text{ sign}(Z_{ij}) = \text{sign}(W_{ij})$$

$$qbits = 15 + \text{floor}\left(\frac{QP}{6}\right)$$

Multiplication factor MF			
QP mod 6	a	b	c
0	13107	5243	8066
1	11916	4660	7490
2	10082	4194	6554
3	9362	3647	5825
4	8192	3355	5243
5	7282	2893	4559

$$MF = \begin{bmatrix} a & c & a & c \\ c & b & c & b \\ a & c & a & c \\ c & b & c & b \end{bmatrix}$$

Quantization offset	
QP	f
0~5	10922
6~11	21845
12~17	43690
18~23	87381
24~29	174762

- Dequantization (or called Rescaling)

After quantization, the **pre-entropy block (Z)** are rescaled to approximate their original values (W'). This is done by multiplying each coefficient with a quantization step size determined by the quantization parameter (QP). **If Intra 16×16 prediction is performed, the 16×16 pre-entropy block must first be divided into sixteen 4×4 blocks, execute this operation for each 4x4 block and then combine them into 16x16 dequantized block (W').** The formula is shown as below:

$$W'_{ij} = Z_{ij} \cdot V_{ij} \cdot 2^{\text{floor}(\frac{QP}{6})}$$

Scaling factor V			
QP mod 6	a	b	c
0	10	16	13
1	11	18	14
2	13	20	16
3	14	23	18
4	16	25	20
5	18	29	23

$$V = \begin{bmatrix} a & c & a & c \\ c & b & c & b \\ a & c & a & c \\ c & b & c & b \end{bmatrix}$$

- Inverse Integer Transform

The **dequantized block (W')** is converted back to the spatial domain using the **inverse 4×4 integer transform**. This step reconstructs an **approximation of the original residual block (X')**, which will be added to the prediction block to form the reconstructed block. **If Intra 16×16 prediction is performed, the 16×16 dequantized block must first be divided into sixteen 4×4 blocks, execute this operation for each 4x4 block and then combine them into 16x16 inverse-transformed block (X').** The formula is shown as below:

$$Y = C_i^T W' C_i, \text{ where } C_i = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \end{bmatrix}$$

$$X' = Y >>> 6 \quad \text{Original formula: } X' = Y/64$$

- Reconstruction

The reconstructed **residual block (X')** is added to the **prediction block** to obtain the final **reconstructed block**. **This block is then used as a reference for predicting subsequent blocks.** The formula is shown as below:

$$\text{Reconstructed}(i, j) = X'(i, j) + \text{Predicted}(i, j) \text{ for each pixel}$$

Note: If Reconstructed (i, j) exceeds 255, set it to 255. If Reconstructed (i, j) is less than 0, set it to 0.

- Rules of input signals

In this exercise, you will first receive the data of **16 frames, numbered from 0 to 15, provided sequentially** at the beginning for each pattern.

- Input Frame

Each frame has **a fixed size of 32×32 pixels**. Each pixel represents the luma (Y) component and is stored as an **8-bit unsigned value**. The 32×32 frame is provided in **raster-scan order**.

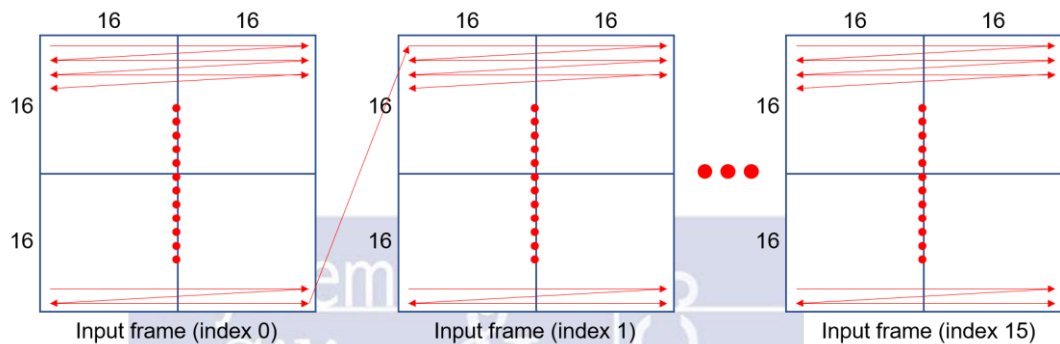


Figure 10. Input order: raster scan order for each 32x32 frame

After that, you will receive several parameters for each set. The following are some rules about the parameters:

- Frame index

The frame index ranges **from 0 to 15** for each set. You must fetch the corresponding 32×32 frame based on this index. **Within each pattern, each index will appear only once and will not be repeated.**

- Prediction mode

There are two prediction modes supported in this lab. The mode value specifies which prediction operation will be applied:

- mode = 1'b0: Intra 16×16 prediction

- mode = 1'b1: Intra 4×4 prediction

You will receive **4 mode values for the 4 16×16 macroblocks and apply the corresponding intra prediction mode to each in raster-scan order**. For example, if the mode values are 1, 0, 0, 1, the top-left and bottom-right macroblocks should use Intra 4×4 prediction, while the others should use Intra 16×16 prediction.

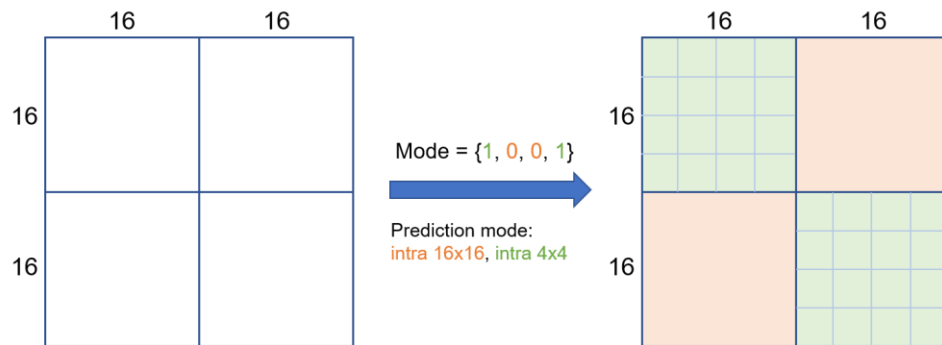


Figure 11. Example for the 4 16x16 frames using different prediction mode

➤ Quantization parameter (QP)

The value of quantization parameter (QP) ranges from 0 to 29 for each set. It can control the compression ratio and quality. Higher QP increases compression but reduces quality, while lower QP preserves detail with less compression.

After 16 sets of parameters, you will get the next pattern.

● Rules of algorithm operation

➤ If intra 4x4 prediction will be applied to 16x16 macroblock, the macroblock will be partitioned into 16 4x4 blocks first and the following steps will be executed:

- Step 1: Intra 4x4 prediction ⇒ obtain 4x4 predict block
- Step 2: Residual computation ⇒ obtain 4x4 residual block
- Step 3: Integer transform ⇒ obtain 4x4 transformed block
- Step 4: Quantization ⇒ obtain 4x4 pre-entropy block
- Step 5: Dequantization ⇒ obtain 4x4 rescaled block
- Step 6: Inverse integer transform ⇒ obtain 4x4 inverse transformed block
- Step 7: Reconstruction ⇒ obtain 4x4 reconstructed block as reference for next round
- Repeat Steps 1–7, a total of 16 rounds so that you can get **16x16 pre-entropy block**

➤ If intra 16x16 prediction will be applied to 16x16 macroblock, the following steps will be executed:

- Step 1: Intra 16x16 prediction ⇒ obtain 16x16 predict block
- Step 2: Residual computation ⇒ obtain 16x16 residual block

- Step 3: Integer transform $\Rightarrow$ obtain 16x16 transformed block
- Step 4: Quantization $\Rightarrow$ obtain **16x16 pre-entropy block**
- Step 5: Dequantization $\Rightarrow$ obtain 16x16 rescaled block
- Step 6: Inverse integer transform $\Rightarrow$ obtain 16x16 inverse transformed block
- Step 7: Reconstruction $\Rightarrow$ obtain 16x16 reconstructed block as reference for next stage

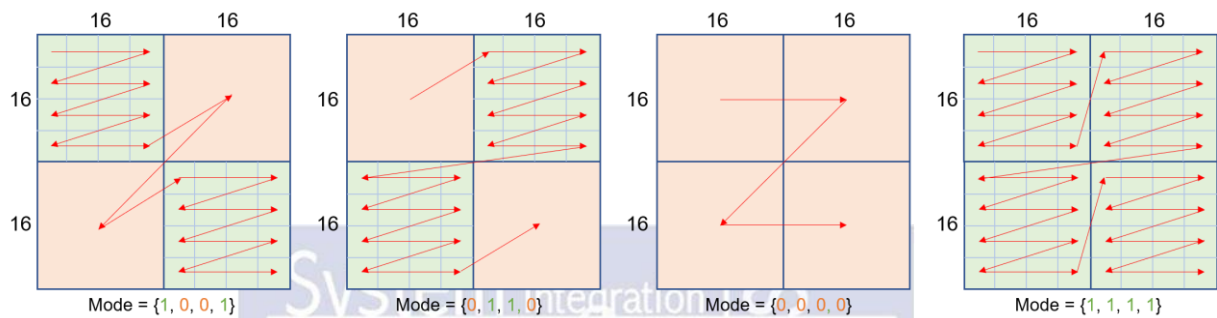


Figure 12. Example for the algorithm operation using different prediction mode

- Rules of output result

You should output the 32x32 pre-entropy frame (Z) by the rules below for each set:

- The output sequence does **not need to be continuous**, but **each set must produce all 1024 output values**.
- The frame is split into four 16x16 macroblocks, each of which is further divided into sixteen 4x4 blocks. Output the 16 pixels of each block in raster-scan order.

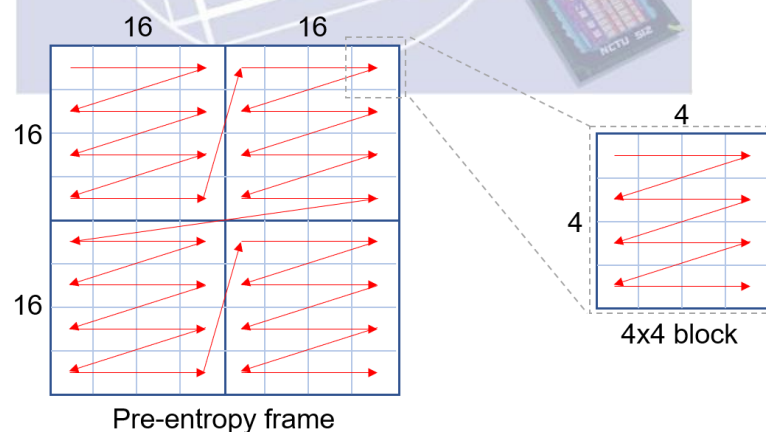


Figure 13. Example for output order

I/O Description

Input	Bit Width	Definition and Description
clk	1	Clock
rst_n	1	Asynchronous active-low reset
in_valid_data	1	High when frame is valid.
data	8	The luma element of the input frame is an 8-bit unsigned value (0~255), sent in raster-scan order continuously when in_valid_data is high.”
in_valid_param	1	High when index , mode and QP is valid.
index	4	Index of the input frame. It will be given in the first cycle when in_valid_param is high. Its value ranges from 0 to 15
mode	1	Prediction mode for a 16×16 macroblock. The signal is provided for 4 cycles when in_valid_param is high. The definitions are as follows: 1'd0 : Intra prediction: 16x16, 1'd1 : Intra prediction: 4x4
QP	5	Quantization parameter can control the compression ratio and quality. Its value ranges from 0 to 29 .
Output	Bit Width	Definition and Description
out_valid	1	High only when out_value is valid. It cannot be overlapped with in_valid_data and in_valid_param . The output sequence does not need to be continuous, but it must be tied high for 1024 cycles in each set.
out_value	32	It will output the final pre-entropy frame (Z) after finishing quantization. The output sequence does not need to be continuous, but each set must produce all 1024 output signed values .

Specification

Top module

1. Top module name: **HLPTE** (file name: **HLPTE.v**)
2. You can adjust your clock period by yourself, but the maximum period is **20 ns**. The precision of clock is 0.1, for example, 19.5 is allowed but 19.55 is not.
3. The execution latency is limited to **10000 cycles for each set**. The latency is the clock cycles between the falling edge of the last cycle of **in_valid_param** and the rising edge of the first cycle of **out_valid**, and must include **all idle cycles between falling and rising edges of out_valid**.
4. The total cell area should not larger than **3,500,000 μm^2** .
5. In this lab, you **must use the memory and generate it yourself**. The number of words and the bits per each word is defined by yourself. The total number and kind of memory is unlimited. We

will check it at **HLPTE.area in 02_SYN/Report**. The area of **Macro/Black Box** must not be 0. The example is shown in following figure.

```
Combinational.area:.....1821995.696653
Buf/Inv.area:.....111973.280126
Noncombinational.area:.....343750.185371
Macro/Black.Box.area:.....214305.703125
Net.Interconnect.area:.....undefined..(No.wire.load.specified)

Total.cell.area:.....2380051.585150
```

6. If any port of memory is connected with mismatched width, the memory will not be synthesized and you will get an error message. Even though the design may still pass gate level simulation, this situation will be regarded as a synthesis fail. In this case, the memory area will be 0 in **HLPTE.area**. We will check it at **syn.log** and **HLPTE.area**.

clock & reset

7. It is **asynchronous reset** and **active-low** architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals that should be reset after the reset signal is asserted.
8. The reset signal (**rst_n**) will be given only once at the beginning of simulation. **All output signals should be reset to 0 after the reset signal is asserted**. The reset duration will last for 3 times the cycle time.
9. The clock (**clk**) will be released after a delay equal to one cycle time following reset deassertion.

in_valid_data & data

10. All input signals are synchronized at negative edge of the clock.
11. **in_valid_data** will come after reset.
12. The input of **frame** is delivered in **raster scan order** for **16384 cycles continuously** when **in_valid_data** is tied high. **When all elements of a frame are delivered completely, it is tied to unknown state**, and **in_valid_data** will also be tied low.

in_valid_param & index, mode and index

13. The input of **mode** is delivered in **raster scan order** for **4 cycles continuously** when **in_valid_param** is tied high. **When all elements of a template are delivered completely, it will be tied to unknown state**.
14. The input of **index** and **QP** will be given in **the first cycle** when the **in_valid_param** is high.
15. **in_valid_param** will be triggered for **a total 16 times** after **in_valid_data** falls in a single pattern. After each time **in_valid_param** triggers, your design will do H.264 Lite pre-entropy

encoding and then **out_valid** will be asserted high for a total of 1024 cycles, which do not need to be continuous.

16. In each pattern, the first **in_valid_param** will come in 2~4 negative edge of clock after **in_valid_data** falls, and the other 15 times **in_valid_param** will come in 2~4 negative edge of clock after **out_valid** has been held high for a total of 1024 cycles and then falls.

out_valid & out_value

17. All output signals should be synchronized at clock positive edge.
18. The TA's pattern will capture your output for checking at clock negative edge.
19. **out_valid** cannot overlap with **in_valid_data** and **in_valid_param** at any time.
20. **out_valid** is limited to be high only when **out_value** is valid.
21. **out_value** should be correct when **out_valid** is high. And **out_value** should be 0 when **out_valid** is low.
22. **out_value** must be delivered for a total of 1024 cycles for each set, which do not need to be continuous, and **out_valid** should be high simultaneously.
23. The next input pattern will come in 2~4 negative edge of clock after **out_valid** has been held high for a total of 1024 cycles and then falls in 16th set.

Synthesis

24. The input delay is set to $0.5 * (\text{clock period})$.
25. The output delay is set to $0.5 * (\text{clock period})$, and the output loading is set to 0.05.
26. The input delay of **clk** and **rst_n** should be 0.
27. The synthesis time should be less than 2 hours.
28. The synthesis result (syn.log) of data type cannot include any latches and error.
29. After synthesis, you can check **HLPTE.area** and **HLPTE.timing**. The area report is valid when the slack in the end of timing report should be non-negative and the result should be MET.

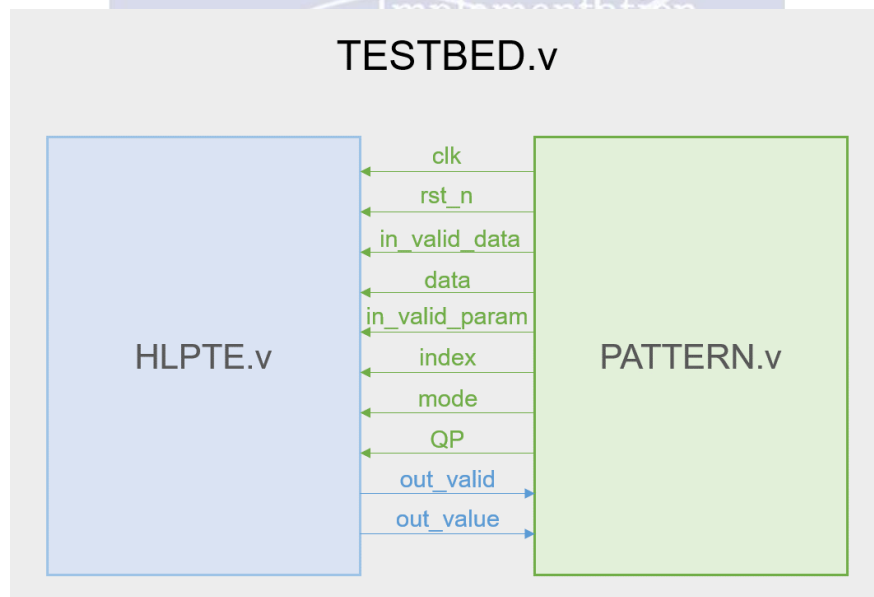
Gate-level simulation

30. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.
31. Error message about SRAM is not allowed during both RTL and gate-level simulation

Supplement

32. In this lab, you are **NOT** allowed to use Designware IP.
33. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.
34. Don't write Chinese or other language comments in the file you sent.
35. Verilog command //synopsys dc_script_begin, //synopsys dc_script_end //synopsys translate_off, //synopsys translate_on are only allowed during the usage of including and setting designware IPs, other design compiler optimizations are forbidden.
36. Using the above commands are allowed, however any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.
37. Any form of display or printing information in verilog design is forbidden. You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Block Diagram



Grading Policy

The performance is determined by the area and latency of your design. The less cost your design has, the higher grade you get.

- Function Validity: 70%
- Performance: 30%
- **Area² * Total latency**

- Total latency = cycle time * latency

Note

- Please submit your design (HLPTE.v) in Lab05/Exercise/09_SUBMIT

- 1st_demo deadline: 2025/10/13 (Mon.) 12:00:00
- 2nd_demo deadline: 2025/10/15 (Wed.) 12:00:00

- Please upload the following files under 09_SUBMIT

- 04_MEM_iclabXXX (Folder)

All memory-related files must be placed directly under this folder. Do not create any subdirectories (e.g., 04_MEM_iclabXXX/04_MEM) to store the files, or your lab will be considered failed.

- {your_cycle_time}_iclabXXX.txt

In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.

- HLPTE_iclabXXX.v

- filelist_iclabXXX.f

- The memory path defined in filelist.f should be related path instead of absolute path.
- No comment contexts are allowed in filelist.f

- Please make sure your file tree structure matches the figure below after you execute the 3 commands: 00_tar, 01_submit and 02_check in 09_SUBMIT sequentially. If your files violate the naming rules or directory structure, your lab will be marked as failed.

```
-- 00_tar ->
-- 01_submit   Linked Path
-- 02_check
-- Lab05_iclabTA07
-- |-- 04_MEM_iclabTA07
-- | |-- memory.v & memory_WC.db
-- | |-- 20_iclabTA07.txt
-- | |-- HLPTE_iclabTA07.v
-- | |-- filelist_iclabTA07.f
-- Lab05_iclabTA07.tar.gz
-- Lab05_iclabTA07_1st_demo.tar.gz
```

- Template folders and reference commands:

- 01_RTL/ (RTL simulation) .01_run_vcs_rtl

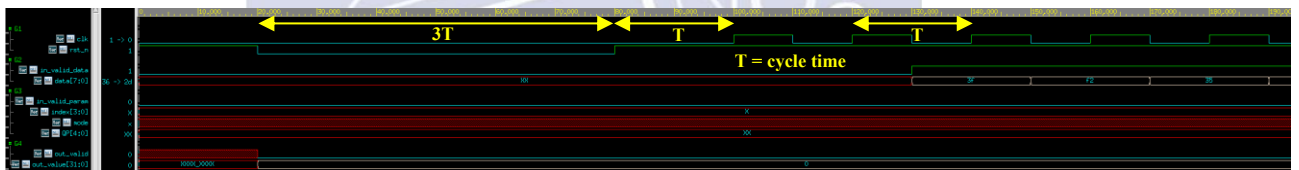
- 02_SYN/ (Synthesis) **./01_run_dc_shell**
 - Check if there is any latch in your design in syn.log
 - Check the timing of design in /Report/HLPTE.timing
 - Check the area of design in /Report/HLPTE.area
- 03_GATE/ (Gate-level simulation) **./01_run_vcs_gate**
- 04_MEM/ (Memory location)
 - You should generate your own memory and put the required files here
- 09_SUBMIT/ (Submit your files) **./00_tar**

./01_submit

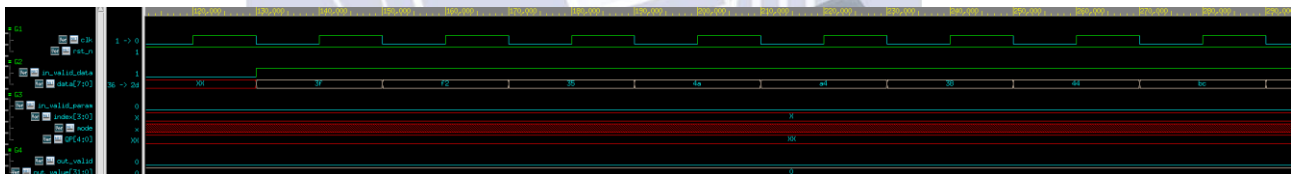
./02_check

Reference Waveform

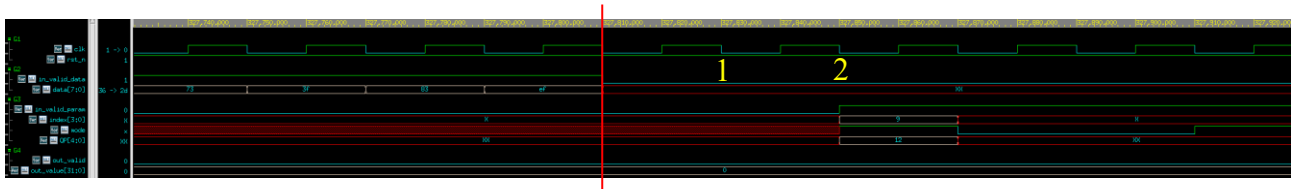
- Asynchronous reset and active-low reset all output to low



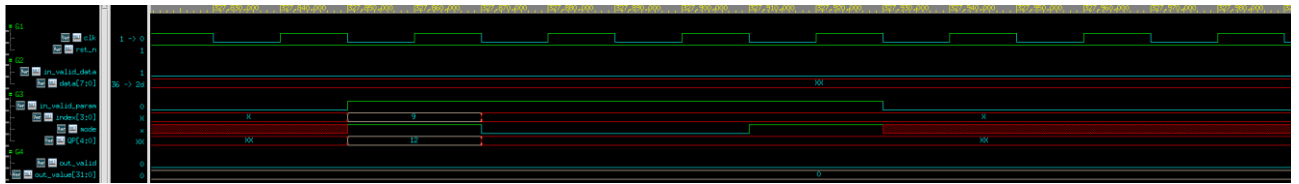
- Input of data when in_valid_data is tied high



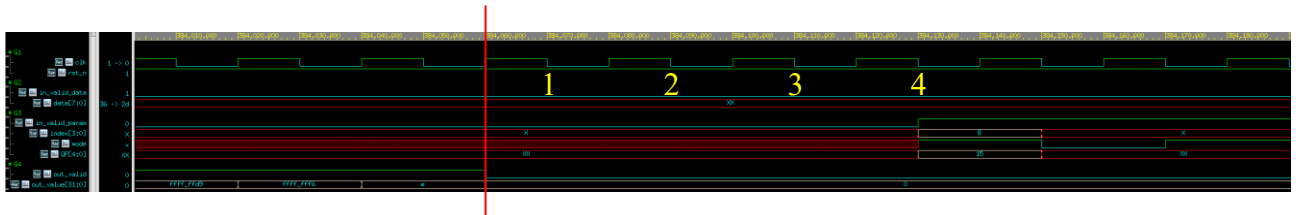
- in_valid_param will come in 2~4 negative edge of clock after in_valid_data is tied low



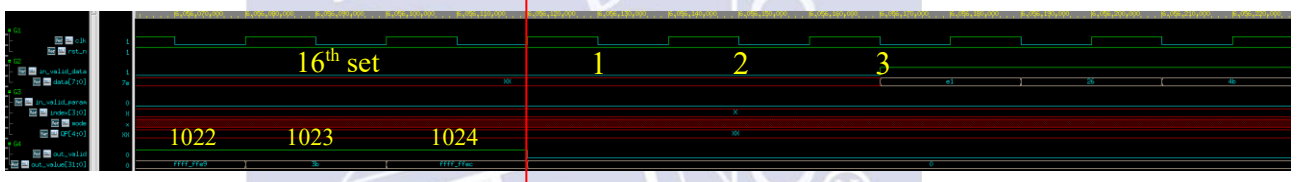
- Input of index, mode and QP when in_valid_param is tied high



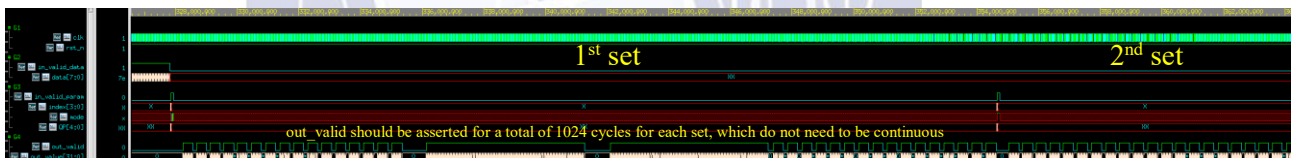
- The other 15 times **in_valid_param** will come in 2~4 negative edge of clock after **out_valid** has been held high for a total of 1024 cycles and then falls in each set



- The next input pattern will come in **2~4 negative edge of clock** after **out_valid** has been held high for a total of 1024 cycles and then falls in 16th set.



- **out_value** must be delivered for a total of 1024 cycles for each set, which do not need to be continuous, and **out_valid** should be high simultaneously.



- The definition of latency

